

Moneyora

Personal Finance & AI Budget Planner

Plan. Track. Thrive.

DATABASE DESIGN DOCUMENT (DBD)

Document Version:	1.0 — Final
Date:	27 June 2026
Based On:	SRS v1.0 + SDD v1.0 (Both Approved)
Database Engine:	SQLite 3.x with SQLCipher AES-256 Encryption
ORM / Access Layer:	sqflite_sqldcipher (Flutter package)
Schema Version:	v1 — Initial Production Schema
Tables:	10 Core Tables + 1 Migration Table
Total Indexes:	12 Performance Indexes
Status:	APPROVED — READY FOR IMPLEMENTATION

Developer: Sanduni • Individual Project

Database Design Document • Phase 3 of Development Lifecycle

TABLE OF CONTENTS

1. INTRODUCTION

- 1.1 Purpose
- 1.2 Scope
- 1.3 Design Principles

2. ENTITY RELATIONSHIP DIAGRAM

- 2.1 Full ERD
- 2.2 Relationship Summary

3. TABLE SCHEMAS

- 3.1 users
- 3.2 accounts
- 3.3 categories
- 3.4 transactions
- 3.5 transfers
- 3.6 recurring_rules
- 3.7 money_plans
- 3.8 plan_allocations
- 3.9 receipt_scans
- 3.10 receipt_items
- 3.11 keyword_dictionary
- 3.12 schema_migrations

4. RELATIONSHIPS & CONSTRAINTS

- 4.1 Foreign Keys
- 4.2 Check Constraints
- 4.3 Unique Constraints

5. INDEXING STRATEGY

- 5.1 Index Definitions
- 5.2 Query-to-Index Mapping

6. DATA FLOW & ACCESS PATTERNS

- 6.1 Data Flow Diagram
- 6.2 Common Query Patterns

7. MIGRATION STRATEGY

- 7.1 Migration Architecture
- 7.2 Migration Scripts v1
- 7.3 Future Migration Template

8. ENCRYPTION & SECURITY

- 8.1 SQLCipher Setup
- 8.2 Key Management

9. DEFAULT SEED DATA

- 9.1 Default Categories
- 9.2 Default Keyword Dictionary Sample

1. INTRODUCTION

1.1 Purpose

This Database Design Document (DBD) defines the complete data storage architecture for **Moneyora**. It specifies all table schemas, relationships, constraints, indexes, migration strategy, encryption approach, and seed data. This document is the authoritative reference for all data layer implementation in Sprint 1 and serves as the foundation for the Domain entities and Repository contracts defined in the SDD v1.0.

1.2 Scope

- 10 core application tables + 1 migration tracking table = **11 tables total**
- 12 performance indexes covering all common query patterns
- Complete foreign key relationships with cascade rules
- AES-256 encryption via SQLCipher on the entire database file
- Versioned migration scripts for every future schema change
- Default seed data: 18 categories + 200+ keyword dictionary entries
- All tables derived directly from SRS v1.0 functional requirements

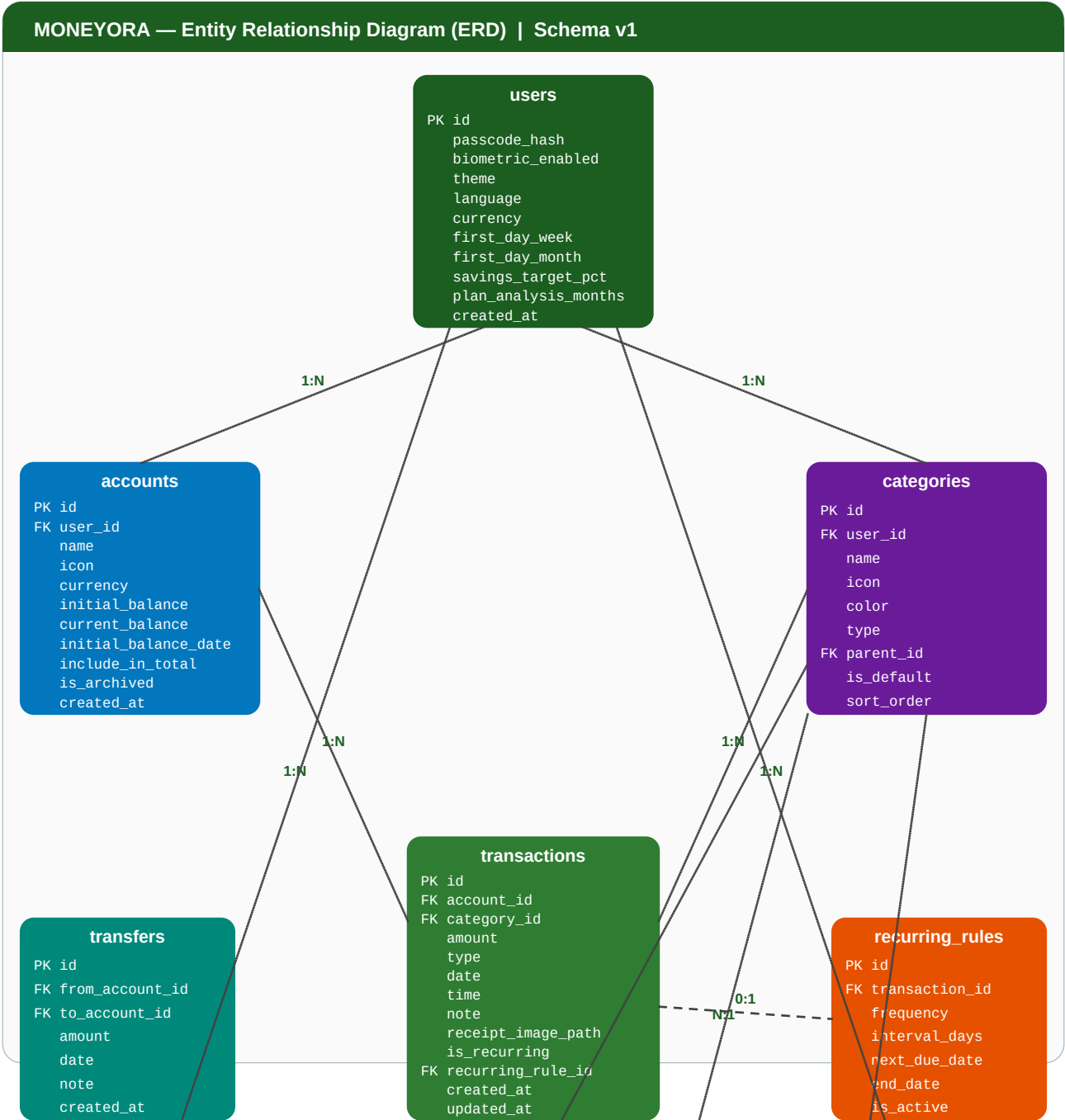
1.3 Design Principles

Principle	Implementation
Offline-First	All data lives locally in SQLite. No cloud dependency for core features.
Encryption by Default	SQLCipher encrypts the entire .db file with AES-256. Applied at open time.
Referential Integrity	All foreign keys enforced with FOREIGN KEY pragma enabled on every connection.
Soft Deletes	No hard deletes for transactions or accounts — is_archived / is_deleted flags preserve history.
Versioned Schema	Every schema change increments the db version. Migration scripts are never destructive.
Audit Timestamps	Every table includes created_at. Mutable tables include updated_at. ISO 8601 text format.
Additive Migrations	ALTER TABLE ADD COLUMN only. No DROP COLUMN, no RENAME TABLE in migrations.

2. ENTITY RELATIONSHIP DIAGRAM

2.1 Full ERD

The diagram below shows all 10 core entities and their relationships. Primary keys are marked * and Foreign keys are marked +.



2.2 Relationship Summary

Parent Table	Child Table	Cardinality	On Delete	Description
users	accounts	1 : N	CASCADE DELETE	One user owns many accounts
money_plans	plan_allocations	1 : N		
plan_allocations	receipt_scans	1 : N		

users	categories	1 : N	CASCADE DELETE	One user owns many categories
users	money_plans	1 : N	CASCADE DELETE	One user owns many plans
users	receipt_scans	1 : N	CASCADE DELETE	One user owns many receipt scans
accounts	transactions	1 : N	RESTRICT	One account has many transactions
categories	transactions	1 : N	RESTRICT	One category has many transactions
transactions	recurring_rules	1 : 0..1	CASCADE DELETE	A transaction may have one recurrence rule
money_plans	plan_allocations	1 : N	CASCADE DELETE	One plan has many category allocations
categories	plan_allocations	1 : N	RESTRICT	One category used in many plan allocations
receipt_scans	receipt_items	1 : N	CASCADE DELETE	One scan produces many line items
categories	receipt_items	1 : N	SET NULL	Category suggested/confirmed for each item
categories	keyword_dictionary	1 : N	CASCADE DELETE	Keywords map to categories
categories	categories	1 : N (self)	SET NULL	Parent-child category hierarchy (max 2 levels)

3. TABLE SCHEMAS

3.1 Table: users

Stores application-level user preferences and security settings. Single-user app — this table always has exactly one row.

TABLE: USERS			
Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Always 1 (single user app)
passcode_hash	TEXT	NULLABLE	SHA-256 hash of PIN (null = no passcode)
biometric_enabled	INTEGER	DEFAULT 0, NOT NULL	1=enabled, 0=disabled
theme	TEXT	DEFAULT 'light', NOT NULL	'light' or 'dark'
language	TEXT	DEFAULT 'en', NOT NULL	ISO 639-1 language code
currency	TEXT	DEFAULT 'LKR', NOT NULL	ISO 4217 currency code
first_day_week	INTEGER	DEFAULT 0, NOT NULL	0=Sunday, 1=Monday
first_day_month	INTEGER	DEFAULT 1, NOT NULL	Day 1-28
savings_target_pct	REAL	DEFAULT 0.0, NOT NULL	Monthly savings target 0.0-100.0
plan_lookback_months	INTEGER	DEFAULT 6, NOT NULL	Months of history for plan generation
created_at	TEXT	NOT NULL	ISO 8601 timestamp of first launch

3.2 Table: accounts

Stores user-defined financial accounts (cash, bank, card, wallet, etc.).

TABLE: ACCOUNTS

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	<i>Auto-generated account ID</i>
user_id	INTEGER	FK users(id), NOT NULL	<i>Owner (always 1 in single-user app)</i>
name	TEXT	NOT NULL	<i>User-defined account name e.g. 'Cash'</i>
icon	TEXT	NOT NULL	<i>Icon identifier string e.g. 'ic_cash'</i>
color	TEXT	DEFAULT '#2E7D32'	<i>Hex color for account display</i>
currency	TEXT	DEFAULT 'LKR', NOT NULL	<i>ISO 4217 code for this account</i>
initial_balance	REAL	DEFAULT 0.0, NOT NULL	<i>Opening balance when account created</i>
current_balance	REAL	DEFAULT 0.0, NOT NULL	<i>Computed balance (updated on each transaction)</i>
initial_balance_date	TEXT	NOT NULL	<i>ISO 8601 date of opening balance</i>
include_in_total	INTEGER	DEFAULT 1, NOT NULL	<i>1=include in combined balance display</i>
is_archived	INTEGER	DEFAULT 0, NOT NULL	<i>1=hidden from active views, 0=active</i>
sort_order	INTEGER	DEFAULT 0	<i>Display order in account list</i>
created_at	TEXT	NOT NULL	<i>ISO 8601 creation timestamp</i>
updated_at	TEXT	NOT NULL	<i>ISO 8601 last update timestamp</i>

3.3 Table: categories

Stores expense and income categories, including defaults and user-created ones. Supports one level of parent-child hierarchy.

TABLE: CATEGORIES

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	<i>Auto-generated category ID</i>
user_id	INTEGER	FK users(id), NOT NULL	<i>Owner user</i>
name	TEXT	NOT NULL	<i>Category name e.g. 'Food', 'Salary'</i>
icon	TEXT	NOT NULL	<i>Icon identifier string</i>
color	TEXT	NOT NULL	<i>Hex color for chart display</i>
type	TEXT	CHECK(type IN ('expense','income'))	<i>Category type</i>
parent_id	INTEGER	FK categories(id), NULLABLE	<i>Parent category (null = top-level)</i>
is_default	INTEGER	DEFAULT 0, NOT NULL	<i>1=system default, 0=user-created</i>
sort_order	INTEGER	DEFAULT 0	<i>Display order within type</i>
created_at	TEXT	NOT NULL	<i>ISO 8601 creation timestamp</i>

3.4 Table: transactions

Central table of the application. Every financial event — expense, income, or transfer — creates one row here. This is the most frequently queried table.

TABLE: TRANSACTIONS

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	<i>Auto-generated ID</i>
account_id	INTEGER	FK accounts(id), NOT NULL	<i>Account this transaction belongs to</i>
category_id	INTEGER	FK categories(id), NOT NULL	<i>Category of this transaction</i>
amount	REAL	NOT NULL, CHECK(amount > 0)	<i>Transaction amount (always positive)</i>
type	TEXT	CHECK(type IN ('expense', 'income', 'transfer'))	<i>Transaction type</i>
date	TEXT	NOT NULL	<i>ISO 8601 date e.g. '2026-06-17'</i>
time	TEXT	NULLABLE	<i>HH:MM time of transaction</i>
note	TEXT	NULLABLE	<i>Optional user note</i>
receipt_image_path	TEXT	NULLABLE	<i>Path to attached receipt image</i>
is_recurring	INTEGER	DEFAULT 0, NOT NULL	<i>1=generated by recurring rule</i>
recurring_rule_id	INTEGER	FK recurring_rules(id), NULLABLE	<i>Source recurring rule (if any)</i>
receipt_batch_id	INTEGER	FK receipt_scans(id), NULLABLE	<i>Receipt scan that created this (if any)</i>
created_at	TEXT	NOT NULL	<i>ISO 8601 insert timestamp</i>
updated_at	TEXT	NOT NULL	<i>ISO 8601 last edit timestamp</i>

3.5 Table: transfers

Records account-to-account transfers. Each transfer also creates two corresponding rows in transactions (one debit, one credit).

TABLE: TRANSFERS

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	<i>Auto-generated transfer ID</i>
from_account_id	INTEGER	FK accounts(id), NOT NULL	<i>Source account (debit)</i>
to_account_id	INTEGER	FK accounts(id), NOT NULL	<i>Destination account (credit)</i>
amount	REAL	NOT NULL, CHECK(amount > 0)	<i>Amount transferred</i>
date	TEXT	NOT NULL	<i>ISO 8601 date</i>
note	TEXT	NULLABLE	<i>Optional note</i>
from_tx_id	INTEGER	FK transactions(id)	<i>Debit transaction row created</i>
to_tx_id	INTEGER	FK transactions(id)	<i>Credit transaction row created</i>
created_at	TEXT	NOT NULL	<i>ISO 8601 creation timestamp</i>

3.6 Table: recurring_rules

Defines recurrence schedules for repeating expenses or income. The background job checks this table daily and creates new transaction rows as needed.

TABLE: RECURRING_RULES

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	<i>Auto-generated ID</i>
template_tx_id	INTEGER	FK transactions(id), NOT NULL	<i>Template transaction to copy</i>
frequency	TEXT	CHECK(frequency IN ('daily', 'weekly', 'monthly', 'yearly', 'custom'))	<i>Recurrence type</i>
interval_days	INTEGER	NULLABLE	<i>Used when frequency='custom'</i>
start_date	TEXT	NOT NULL	<i>First occurrence date</i>
end_date	TEXT	NULLABLE	<i>Stop after this date (null=forever)</i>
next_due_date	TEXT	NOT NULL	<i>Date of next transaction to generate</i>
is_active	INTEGER	DEFAULT 1, NOT NULL	<i>1=active, 0=pended</i>
created_at	TEXT	NOT NULL	<i>ISO 8601 creation timestamp</i>

3.7 Table: money_plans

Stores user-created budget plans. Each plan has a time period and a set of per-category allocations stored in plan_allocations.

TABLE: MONEY_PLANS

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-generated plan ID
user_id	INTEGER	FK users(id), NOT NULL	Owner user
name	TEXT	NOT NULL	Plan name e.g. 'June Monthly Plan'
period_type	TEXT	CHECK(period_type IN ('day','week','month','year','custom'))	Time period type
start_date	TEXT	NOT NULL	Plan start date ISO 8601
end_date	TEXT	NOT NULL	Plan end date ISO 8601
total_budget	REAL	NULLABLE	Total budget cap (null=sum of allocations)
is_active	INTEGER	DEFAULT 0, NOT NULL	1=currently active plan, 0=saved/archived
created_at	TEXT	NOT NULL	ISO 8601 creation timestamp

3.8 Table: plan_allocations

One row per category per plan. Stores the AI-generated (or user-adjusted) budget allocation and tracks actual spending for real-time plan vs. actual comparison.

TABLE: PLAN_ALLOCATIONS

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-generated ID
plan_id	INTEGER	FK money_plans(id), NOT NULL	Parent plan
category_id	INTEGER	FK categories(id), NOT NULL	Category being allocated
allocated_amount	REAL	NOT NULL, CHECK(allo cated_amount >= 0)	Budget allocated for this category
spent_amount	REAL	DEFAULT 0.0, NOT NULL	Actual spent (updated on each transaction)
confidence_level	TEXT	CHECK(confidence_leve l IN ('High', 'Medium', 'Low'))	AI prediction confidence
expense_type	TEXT	CHECK(expense_type IN ('Fixed', 'Variable', 'Sea sonal'))	AI classification
is_user_modified	INTEGER	DEFAULT 0, NOT NULL	1=user manually adjusted this allocation
notes	TEXT	NULLABLE	AI suggestion notes for this category

3.9 Table: receipt_scans

Stores metadata for each scanned receipt. The actual image is stored as an encrypted file on disk; only the path is stored here.

TABLE: RECEIPT_SCANS

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-generated scan ID
user_id	INTEGER	FK users(id), NOT NULL	Owner user
image_path	TEXT	NOT NULL	Encrypted file path on device
merchant_name	TEXT	NULLABLE	Detected merchant/store name
receipt_date	TEXT	NULLABLE	Date printed on receipt (ISO 8601)
total_amount	REAL	NULLABLE	Total amount detected on receipt
tax_amount	REAL	NULLABLE	Tax / VAT amount detected
confidence_score	INTEGER	CHECK(confidence_scor e BETWEEN 0 AND 100)	Overall OCR confidence 0-100
status	TEXT	CHECK(status IN ('pendi ng', 'confirmed', 'rejected'))	Scan confirmation status
item_count	INTEGER	DEFAULT 0, NOT NULL	Number of line items detected
created_at	TEXT	NOT NULL	ISO 8601 scan timestamp

3.10 Table: receipt_items

Stores each line item extracted from a receipt scan. Each confirmed item becomes one row in transactions.

TABLE: RECEIPT_ITEMS

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-generated item ID
receipt_scan_id	INTEGER	FK receipt_scans(id), NOT NULL	Parent receipt scan
name	TEXT	NOT NULL	Item description from OCR
quantity	REAL	DEFAULT 1.0, NOT NULL	Quantity (e.g. 2 for 2x items)
unit_price	REAL	NULLABLE	Price per unit (if detected)
total_price	REAL	NOT NULL, CHECK(total_price >= 0)	Line total for this item
suggested_category_id	INTEGER	FK categories(id), NULLABLE	AI-suggested category
confirmed_category_id	INTEGER	FK categories(id), NULLABLE	User-confirmed final category
confidence_score	INTEGER	CHECK(confidence_score BETWEEN 0 AND 100)	Categorization confidence
transaction_id	INTEGER	FK transactions(id), NULLABLE	Created transaction (after confirmation)
is_discarded	INTEGER	DEFAULT 0, NOT NULL	1=user removed this item

3.11 Table: keyword_dictionary

Maps keywords and merchant names to categories. Pre-populated with 200+ entries at first launch. Updated when users correct AI categorization suggestions.

TABLE: KEYWORD_DICTIONARY

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	<i>Auto-generated ID</i>
keyword	TEXT	NOT NULL	<i>Keyword to match (lowercase)</i>
category_id	INTEGER	FK categories(id), NOT NULL	<i>Target category for this keyword</i>
match_type	TEXT	CHECK(match_type IN ('exact','contains','startswith'))	<i>How to match the keyword</i>
priority	INTEGER	DEFAULT 5, CHECK(priority BETWEEN 1 AND 10)	<i>Match priority — user entries=10</i>
usage_count	INTEGER	DEFAULT 0, NOT NULL	<i>Times this mapping was applied</i>
is_user_defined	INTEGER	DEFAULT 0, NOT NULL	<i>1=user-created, 0=system default</i>
created_at	TEXT	NOT NULL	<i>ISO 8601 creation timestamp</i>

3.12 Table: schema_migrations

Tracks applied database migrations. Prevents re-running migrations on app updates.

TABLE: SCHEMA_MIGRATIONS

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	<i>Auto-generated ID</i>
version	INTEGER	UNIQUE, NOT NULL	<i>Schema version number (matches db version)</i>
description	TEXT	NOT NULL	<i>Human-readable migration description</i>
applied_at	TEXT	NOT NULL	<i>ISO 8601 timestamp when migration ran</i>
checksum	TEXT	NULLABLE	<i>SHA-256 of migration SQL for integrity check</i>

4. RELATIONSHIPS & CONSTRAINTS

4.1 Foreign Key Configuration

SQLite does **not** enforce foreign keys by default. The following PRAGMA must be executed on every database connection open:

```
PRAGMA foreign_keys = ON;  
PRAGMA journal_mode = WAL; -- Write-Ahead Logging for better concurrency  
PRAGMA synchronous = NORMAL; -- Balance between safety and performance
```

4.2 Check Constraints Summary

Column	Constraint
transactions.type	IN ('expense', 'income', 'transfer')
transactions.amount	> 0
categories.type	IN ('expense', 'income')
plan_allocations.allocated_amount	>= 0
plan_allocations.confidence_level	IN ('High', 'Medium', 'Low')
plan_allocations.expense_type	IN ('Fixed', 'Variable', 'Seasonal')
receipt_scans.confidence_score	BETWEEN 0 AND 100
receipt_scans.status	IN ('pending', 'confirmed', 'rejected')
receipt_items.total_price	>= 0
receipt_items.confidence_score	BETWEEN 0 AND 100
keyword_dictionary.match_type	IN ('exact', 'contains', 'startswith')
keyword_dictionary.priority	BETWEEN 1 AND 10
recurring_rules.frequency	IN ('daily', 'weekly', 'monthly', 'yearly', 'custom')
transfers.amount	> 0
users.first_day_week	IN (0, 1)
users.first_day_month	BETWEEN 1 AND 28

4.3 Unique Constraints

Table	Unique Columns	Reason
accounts	user_id + name	No duplicate account names per user

categories	user_id + name + type	No duplicate category names per type per user
money_plans	user_id + name	No duplicate plan names per user
keyword_dictionary	keyword + category_id	No duplicate keyword-category mappings
schema_migrations	version	Each migration version applied only once
plan_allocations	plan_id + category_id	One allocation per category per plan

5. INDEXING STRATEGY

5.1 Index Definitions

All indexes are created in the initial schema migration (version 1). Indexes target the most frequent query patterns identified from the SRS.

```
-- Core transaction query indexes
CREATE INDEX idx_tx_date ON transactions(date);
CREATE INDEX idx_tx_account_date ON transactions(account_id, date);
CREATE INDEX idx_tx_category ON transactions(category_id);
CREATE INDEX idx_tx_type_date ON transactions(type, date);
CREATE INDEX idx_tx_batch ON transactions(receipt_batch_id);

-- Account & category indexes
CREATE INDEX idx_accounts_user ON accounts(user_id, is_archived);
CREATE INDEX idx_cat_user_type ON categories(user_id, type);

-- Plan tracking indexes
CREATE INDEX idx_plan_alloc_plan ON plan_allocations(plan_id);
CREATE INDEX idx_plan_alloc_cat ON plan_allocations(category_id);

-- Receipt scanner indexes
CREATE INDEX idx_receipt_items ON receipt_items(receipt_scan_id);
CREATE INDEX idx_receipt_status ON receipt_scans(status, created_at);

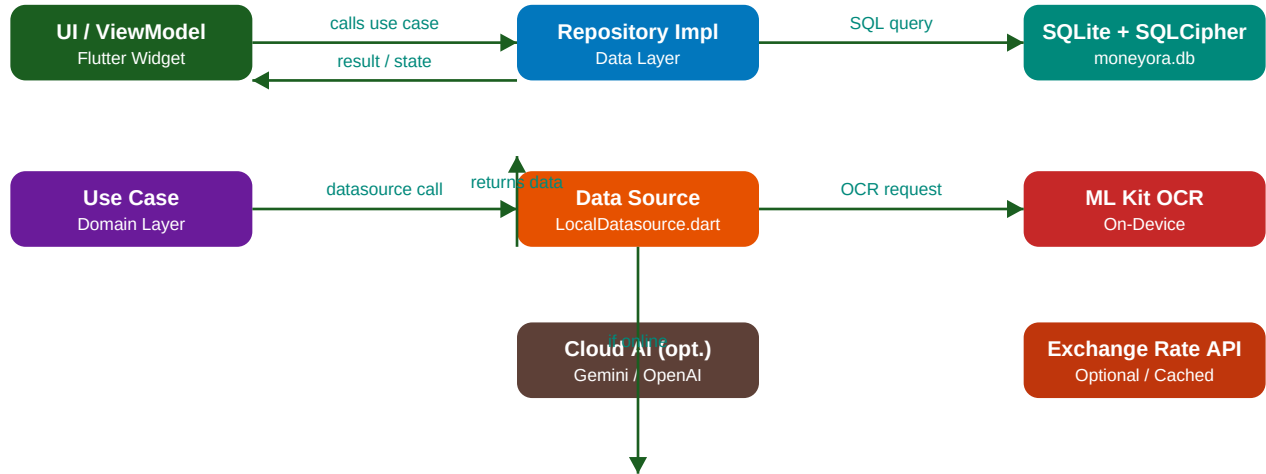
-- Keyword lookup index (critical for OCR speed)
CREATE INDEX idx_keyword_lookup ON keyword_dictionary(keyword, priority DESC);
```

5.2 Query-to-Index Mapping

Query	Index Used	Target Performance
Home screen chart (date range)	idx_tx_account_date	< 5 ms for 10k rows
Category totals for pie chart	idx_tx_category	< 5 ms for 10k rows
Transaction list (filtered by date)	idx_tx_date	< 3 ms for 10k rows
Expense vs income split	idx_tx_type_date	< 5 ms for 10k rows
Money plan analysis (6 months)	idx_tx_category + idx_tx_date	< 50 ms for 10k rows
Plan vs actual tracking	idx_plan_alloc_plan	< 2 ms
Receipt items for a scan	idx_receipt_items	< 1 ms
Keyword categorization (OCR)	idx_keyword_lookup	< 1 ms per keyword
Account balance calculation	idx_accounts_user	< 2 ms

6. DATA FLOW & ACCESS PATTERNS

6.1 Data Flow Diagram



Core path (UI → UseCase → Repository → SQLite) is 100%

Figure 2: Data Flow — offline core path (solid) + optional online path (dashed)

6.2 Common Query Patterns

Add Expense

```
BEGIN TRANSACTION;
INSERT INTO transactions (account_id, category_id, amount, type, date, ...) VALUES (...);
UPDATE accounts SET current_balance = current_balance - ? WHERE id = ?;
UPDATE plan_allocations SET spent_amount = spent_amount + ?
  WHERE plan_id = (SELECT id FROM money_plans WHERE is_active=1)
  AND category_id = ?;
COMMIT;
```

Home Screen Chart Data

```
SELECT c.name, c.color, c.icon, SUM(t.amount) as total
FROM transactions t JOIN categories c ON t.category_id = c.id
WHERE t.account_id = ? AND t.type = 'expense'
  AND t.date BETWEEN ? AND ?
GROUP BY t.category_id
ORDER BY total DESC;
```

Money Plan Analysis

```
SELECT t.category_id,
       AVG(monthly_total) as avg_monthly,
       (strftime('%Y-%m', t.date)) as month
FROM transactions t
WHERE t.type='expense'
  AND t.date >= date('now', '-6 months')
GROUP BY t.category_id, strftime('%Y-%m', t.date)
ORDER BY t.category_id, month;
```

Plan vs Actual Progress

```
SELECT pa.category_id, c.name, c.icon,  
       pa.allocated_amount,  
       pa.spent_amount,  
       ROUND(pa.spent_amount * 100.0 / pa.allocated_amount, 1) as pct_used,  
       pa.confidence_level  
FROM plan_allocations pa  
JOIN categories c ON pa.category_id = c.id  
WHERE pa.plan_id = ?  
ORDER BY pct_used DESC;
```

Receipt OCR Keyword Lookup

```
SELECT kd.category_id, kd.priority, c.name  
FROM keyword_dictionary kd  
JOIN categories c ON kd.category_id = c.id  
WHERE (kd.match_type = 'exact'      AND kd.keyword = lower(?))  
      OR (kd.match_type = 'contains' AND lower(?) LIKE '%' || kd.keyword || '%')  
      OR (kd.match_type = 'startswith' AND lower(?) LIKE kd.keyword || '%')  
ORDER BY kd.priority DESC  
LIMIT 1;
```

7. MIGRATION STRATEGY

7.1 Migration Architecture

- The SQLite `user_version` PRAGMA tracks the current schema version
- On every app launch, `DatabaseHelper.onUpgrade()` is called if version increased
- Each migration is a single Dart function: `Future migrateVxToVy(Database db)`
- Migrations are **additive only** — never DROP COLUMN or DROP TABLE
- Every migration inserts a row into `schema_migrations` upon completion
- Migrations run in sequence — `v1→v2→v3`, never `v1→v3` directly

7.2 Migration Script v1 — Initial Schema

```
// migration_v1.dart – Creates all tables and indexes

Future migrateV0ToV1(Database db) async {
  // Enable foreign keys
  await db.execute('PRAGMA foreign_keys = ON;');
  await db.execute('PRAGMA journal_mode = WAL;');

  // Create all tables
  await db.execute('''
    CREATE TABLE IF NOT EXISTS users (
      id INTEGER PRIMARY KEY,
      passcode_hash TEXT,
      biometric_enabled INTEGER DEFAULT 0 NOT NULL,
      theme TEXT DEFAULT 'light' NOT NULL,
      language TEXT DEFAULT 'en' NOT NULL,
      currency TEXT DEFAULT 'LKR' NOT NULL,
      first_day_week INTEGER DEFAULT 0 NOT NULL
        CHECK(first_day_week IN (0,1)),
      first_day_month INTEGER DEFAULT 1 NOT NULL
        CHECK(first_day_month BETWEEN 1 AND 28),
      savings_target_pct REAL DEFAULT 0.0 NOT NULL,
      plan_lookback_months INTEGER DEFAULT 6 NOT NULL,
      created_at TEXT NOT NULL
    );
  ''');

  // ... (all CREATE TABLE statements) ...

  // Create all indexes
  await db.execute(
    'CREATE INDEX idx_tx_date ON transactions(date);');
  // ... (all CREATE INDEX statements) ...

  // Insert seed data
  await _seedDefaultUser(db);
  await _seedDefaultCategories(db);
  await _seedKeywordDictionary(db);

  // Record migration
  await db.insert('schema_migrations', {
    'version': 1,
    'description': 'Initial schema – all tables, indexes, seed data',
    'applied_at': DateTime.now().toIso8601String(),
  });
}
```

7.3 Future Migration Template

```
// migration_v2.dart – Example: adding a new column

Future migrateV1ToV2(Database db) async {
  // SAFE: ADD COLUMN is always backward compatible
  await db.execute(
    'ALTER TABLE transactions ADD COLUMN tags TEXT;'
  );

  // Record migration
  await db.insert('schema_migrations', {
    'version': 2,
    'description': 'Added tags column to transactions',
    'applied_at': DateTime.now().toIso8601String(),
  });
}

// NEVER do this:
// await db.execute('DROP COLUMN xyz');      // FORBIDDEN
// await db.execute('DROP TABLE accounts'); // FORBIDDEN
// await db.execute('ALTER TABLE RENAME');  // FORBIDDEN in migrations
```

8. ENCRYPTION & SECURITY

8.1 SQLCipher Setup

The entire SQLite database file is encrypted with **AES-256-CBC** via SQLCipher. Encryption is transparent — all SQL queries work identically. The encryption key is derived and stored securely, never written to plain storage.

```
// pubspec.yaml
dependencies:
  sqflite_sqlcipher: ^2.2.0
  flutter_secure_storage: ^9.0.0

// DatabaseHelper.dart
import 'package:sqflite_sqlcipher/sqflite.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

Future _openEncryptedDB() async {
  final storage = FlutterSecureStorage();
  String? key = await storage.read(key: 'moneyora_db_key');
  if (key == null) {
    final bytes = List.generate(32, (_) => Random.secure().nextInt(256));
    key = base64Url.encode(bytes);
    await storage.write(key: 'moneyora_db_key', value: key);
  }
  final dbPath = await getDatabasesPath();
  return openDatabase(
    join(dbPath, 'moneyora.db'),
    password: key,          // <-- SQLCipher AES-256 key
    version: 1,
    onCreate: _onCreate,
    onUpgrade: _onUpgrade,
    onOpen: (db) async {
      await db.execute('PRAGMA foreign_keys = ON;');
      await db.execute('PRAGMA journal_mode = WAL;');
    },
  );
}
```

8.2 Key Management

Aspect	Implementation
Key Generation	32 random bytes via Dart's Random.secure() — cryptographically secure
Key Storage	iOS: Keychain (via flutter_secure_storage) — Android: Android Keystore
Key Encoding	Base64URL encoded before storage — URL-safe, no special characters
Key Rotation	Not implemented in v1 — planned for v2 (requires re-keying the DB)
Key Backup	Key is NOT included in data backups — intentional (backup is useless without device)
Lost Key Recovery	If key is lost (factory reset without backup), data is unrecoverable — by design

9. DEFAULT SEED DATA

9.1 Default Categories

Inserted by `_seedDefaultCategories()` on first launch (migration v1). These are the 18 default categories matching the SRS FR-EXP-003 and FR-INC-002 requirements.

ID	Name	Type	Icon Key	Default Color	is_default
1	Bills	expense	ic_bills	#E53935	1
2	Car	expense	ic_car	#FB8C00	1
3	Clothes	expense	ic_clothes	#8E24AA	1
4	Communications	expense	ic_phone	#039BE5	1
5	Eating Out	expense	ic_restaurant	#F4511E	1
6	Entertainment	expense	ic_entertainment	#E91E63	1
7	Food	expense	ic_food	#43A047	1
8	Gifts	expense	ic_gift	#FFB300	1
9	Health	expense	ic_health	#E53935	1
10	House	expense	ic_house	#3949AB	1
11	Pets	expense	ic_pet	#00897B	1
12	Sports	expense	ic_sports	#7CB342	1
13	Taxi	expense	ic_taxi	#FFD600	1
14	Toiletry	expense	ic_toiletry	#26C6DA	1
15	Transport	expense	ic_transport	#5E35B1	1
16	Deposits	income	ic_deposit	#2E7D32	1
17	Salary	income	ic_salary	#1B5E20	1
18	Savings	income	ic_piggy	#00695C	1

9.2 Default Keyword Dictionary Sample

200+ keyword entries are seeded on first launch. Below is a representative sample. The full list is defined in `lib/core/database/seed/keyword_seed.dart`.

Keyword	Maps To Category	Match Type	Priority
rice	Food	contains	5
bread	Food	contains	5

milk	Food	contains	5
noodles	Food	contains	5
restaurant	Eating Out	contains	5
kfc	Eating Out	exact	5
mcdonalds	Eating Out	contains	5
petrol	Car	contains	5
fuel	Car	contains	5
diesel	Car	contains	5
bus	Transport	exact	5
train	Transport	contains	5
uber	Taxi	exact	5
pickme	Taxi	exact	5
panadol	Health	contains	5
pharmacy	Health	contains	5
medicine	Health	contains	5
shampoo	Toiletry	contains	5
soap	Toiletry	contains	5
electricity	Bills	contains	5
water bill	Bills	contains	5
netflix	Entertainment	exact	5
spotify	Entertainment	exact	5
dialog	Communications	exact	5
mobitel	Communications	exact	5
salary	Salary	contains	5
deposit	Deposits	contains	5
savings	Savings	contains	5